Principles of Programming Languages
Lecture 10: Paradigms: OOP.

Andrei Arusoaie[1]

[1]Department of Computer Science

December 13, 2016

Outline

Outline

Outline

Paradigms

- Imperative Programming: ✓

Paradigms

- Imperative Programming: ✓
- Object Oriented Programming

Paradigms

- Imperative Programming: ✓
- Object Oriented Programming
- Functional Programming

Paradigms

- Imperative Programming: ✓
- Object Oriented Programming
- Functional Programming
- Logic Programming

Object-Oriented Programming: Motivation-1

- **Complex** software

Object-Oriented Programming: Motivation-1

- **Complex** software
- NATO conference, 1968: *software crisis*
  1. User dissatisfaction with the final product

Object-Oriented Programming: Motivation-1

- **Complex** software
- NATO conference, 1968: *software crisis*
  1. User dissatisfaction with the final product
  2. cost overruns

Object-Oriented Programming: Motivation-1

- **Complex** software
- NATO conference, 1968: *software crisis*
  1. User dissatisfaction with the final product
  2. cost overruns
  3. buggy software

Object-Oriented Programming: Motivation-1

- **Complex** software
- NATO conference, 1968: *software crisis*
  1. User dissatisfaction with the final product
  2. cost overruns
  3. buggy software
  4. brittle software

Motivation-2

Main issues in large software projects:

- Names: functions, variables, ...

Motivation-2

Main issues in large software projects:

- Names: functions, variables, ...
- Constraints: time and space complexity

Motivation-2

Main issues in large software projects:

- Names: functions, variables, ...
- Constraints: time and space complexity
- Memory management (garbage collection and address spaces)

Motivation-2

Main issues in large software projects:

- Names: functions, variables, ...
- Constraints: time and space complexity
- Memory management (garbage collection and address spaces)
- Concurrency

Motivation-2

Main issues in large software projects:

- Names: functions, variables, ...
- Constraints: time and space complexity
- Memory management (garbage collection and address spaces)
- Concurrency
- Event-driven interfaces

Motivation-2

Main issues in large software projects:

- Names: functions, variables, ...
- Constraints: time and space complexity
- Memory management (garbage collection and address spaces)
- Concurrency
- Event-driven interfaces

How to fix all these?

How to fix all these?

- Extreme discipline

How to fix all these?

- Extreme discipline (hard to get when working in teams)

How to fix all these?

- Extreme discipline (hard to get when working in teams)
- Or a new paradigm that facilitates:

How to fix all these?

- Extreme discipline (hard to get when working in teams)
- Or a new paradigm that facilitates:
  1. solving very complex problems

How to fix all these?

- Extreme discipline (hard to get when working in teams)
- Or a new paradigm that facilitates:
  1. solving very complex problems
  2. working in teams

How to fix all these?

- Extreme discipline (hard to get when working in teams)
- Or a new paradigm that facilitates:
  1. solving very complex problems
  2. working in teams
  3. code modularity

How to fix all these?

- Extreme discipline (hard to get when working in teams)
- Or a new paradigm that facilitates:
  1. solving very complex problems
  2. working in teams
  3. code modularity
  4. easy development and code maintenance

Object-Oriented Programming

A bunch of concepts:

Object-Oriented Programming

A bunch of concepts:

- Classes, objects, members (attributes + methods), access modifiers

Object-Oriented Programming

A bunch of concepts:

- Classes, objects, members (attributes + methods), access modifiers
- Constructors, destructors

Object-Oriented Programming

A bunch of concepts:

- Classes, objects, members (attributes + methods), access modifiers
- Constructors, destructors
- Overloading, overriding, shadowing

Object-Oriented Programming

A bunch of concepts:

- Classes, objects, members (attributes + methods), access modifiers
- Constructors, destructors
- Overloading, overriding, shadowing
- *Encapsulation*, information hidding

Object-Oriented Programming

A bunch of concepts:

- Classes, objects, members (attributes + methods), access modifiers
- Constructors, destructors
- Overloading, overriding, shadowing
- *Encapsulation*, information hidding
- *Abstraction*

Object-Oriented Programming

A bunch of concepts:

- Classes, objects, members (attributes + methods), access modifiers
- Constructors, destructors
- Overloading, overriding, shadowing
- *Encapsulation*, information hidding
- *Abstraction*
- *Inheritance*

Object-Oriented Programming

A bunch of concepts:

- Classes, objects, members (attributes + methods), access modifiers
- Constructors, destructors
- Overloading, overriding, shadowing
- *Encapsulation*, information hidding
- *Abstraction*
- *Inheritance*
- *Polymorphism*

Object-Oriented Programming

A bunch of concepts:

- Classes, objects, members (attributes + methods), access modifiers
- Constructors, destructors
- Overloading, overriding, shadowing
- *Encapsulation*, information hidding
- *Abstraction*
- *Inheritance*
- *Polymorphism*
- Dynamic method selection (or dynamic dispatch)

Object-Oriented Programming

A bunch of concepts:

- Classes, objects, members (attributes + methods), access modifiers
- Constructors, destructors
- Overloading, overriding, shadowing
- *Encapsulation*, information hidding
- *Abstraction*
- *Inheritance*
- *Polymorphism*
- Dynamic method selection (or dynamic dispatch)
- Interfaces, abstract classes

Classes vs. Objects

- Class =

Classes vs. Objects

- Class = a specification used to *construct* objects

Classes vs. Objects

- Class = a specification used to *construct* objects
- Classes establish the name of the class, the *data method* members with types and signature, visibility and implementation.

Classes vs. Objects

- Class = a specification used to *construct* objects
- Classes establish the name of the class, the *data method* members with types and signature, visibility and implementation.
- Object =

Classes vs. Objects

- Class = a specification used to *construct* objects
- Classes establish the name of the class, the *data method* members with types and signature, visibility and implementation.
- Object = an *instance* of a class; it has a state where its *data members* (or *attributes* or *properties*) – described in the corresponding class – have values

Constructors and Destructors

- *Constructor*

Constructors and Destructors

- *Constructor*: a special “method” which has the same name as the class and no return type

Constructors and Destructors

- *Constructor*: a special “method” which has the same name as the class and no return type
- When an object is created the *constructor* is called: allocate the necessary memory, initialise members (do not forget inheritance)

Constructors and Destructors

- *Constructor*: a special “method” which has the same name as the class and no return type
- When an object is created the *constructor* is called: allocate the necessary memory, initialise members (do not forget inheritance)
- Typically (in many programming language) we use the `new` keyword to call the constructor

Constructors and Destructors

- *Constructor*: a special “method” which has the same name as the class and no return type
- When an object is created the *constructor* is called: allocate the necessary memory, initialise members (do not forget inheritance)
- Typically (in many programming language) we use the `new` keyword to call the constructor
- *Destructor*

Constructors and Destructors

- *Constructor*: a special “method” which has the same name as the class and no return type
- When an object is created the *constructor* is called: allocate the necessary memory, initialise members (do not forget inheritance)
- Typically (in many programming language) we use the `new` keyword to call the constructor
- *Destructor*: frees the memory occupied by the object

Constructors and Destructors

- *Constructor*: a special “method” which has the same name as the class and no return type
- When an object is created the *constructor* is called: allocate the necessary memory, initialise members (do not forget inheritance)
- Typically (in many programming language) we use the `new` keyword to call the constructor
- *Destructor*: frees the memory occupied by the object
- *Garbage Collector*

Constructors and Destructors

- *Constructor*: a special “method” which has the same name as the class and no return type
- When an object is created the *constructor* is called: allocate the necessary memory, initialise members (do not forget inheritance)
- Typically (in many programming language) we use the `new` keyword to call the constructor
- *Destructor*: frees the memory occupied by the object
- *Garbage Collector*: automatic memory management (efficient memory allocation, automatically frees the memory, enables memory reuse)

Encapsulation

- *Encapsulation*

Encapsulation

- *Encapsulation*: organise the data and operations (over that data) in a single unit

Encapsulation

- *Encapsulation*: organise the data and operations (over that data) in a single unit
- Advantage: manipulate the objects in an uniform and consistent way using methods

Encapsulation

- *Encapsulation*: organise the data and operations (over that data) in a single unit
- Advantage: manipulate the objects in an uniform and consistent way using methods
- *Information hidding*: separate the interface (i.e., the public information in a class) from the implementation

Encapsulation

- *Encapsulation*: organise the data and operations (over that data) in a single unit
- Advantage: manipulate the objects in an uniform and consistent way using methods
- *Information hidding*: separate the interface (i.e., the public information in a class) from the implementation
- *Access modifiers*: used to restrict/grant access to object's data

Abstraction

- It comes in the same “package” with *Encapsulation*

Abstraction

- It comes in the same “package” with *Encapsulation*
- Example: driving a car doesn’t require knowing the internals of the combustion engine

Abstraction

- It comes in the same “package” with *Encapsulation*
- Example: driving a car doesn’t require knowing the internals of the combustion engine
- *Abstraction*: preserve the information that is important in a given context

Abstraction

- It comes in the same “package” with *Encapsulation*
- Example: driving a car doesn’t require knowing the internals of the combustion engine
- *Abstraction*: preserve the information that is important in a given context
- The human mind can operate easier with abstractions

Abstraction

- It comes in the same “package” with *Encapsulation*
- Example: driving a car doesn’t require knowing the internals of the combustion engine
- *Abstraction*: preserve the information that is important in a given context
- The human mind can operate easier with abstractions
- Experiment:

Abstraction

- It comes in the same “package” with *Encapsulation*
- Example: driving a car doesn’t require knowing the internals of the combustion engine
- *Abstraction*: preserve the information that is important in a given context
- The human mind can operate easier with abstractions
- Experiment:1973286437914682519732864379146825197328643791468251973286437914682 (30 sec)

Abstraction

- It comes in the same “package” with *Encapsulation*
- Example: driving a car doesn’t require knowing the internals of the combustion engine
- *Abstraction*: preserve the information that is important in a given context
- The human mind can operate easier with abstractions
- Experiment:1973286437914682519732864379146 82 (30 sec)
  1. How many can you remember?
  2. How many can you remember with unlimited amount of time?

Polymorphism

- It comes in many flavours:

Polymorphism

- It comes in many flavours:
  1. Overloading: multiple implementations of the same method with different parameters

Polymorphism

- It comes in many flavours:
  1. Overloading: multiple implementations of the same method with different parameters
  2. Parametric – often used with *generics*: methods work with parametric classes (e.g., `sort` for `List<T>`)

Polymorphism

- It comes in many flavours:
  1. Overloading: multiple implementations of the same method with different parameters
  2. Parametric – often used with *generics*: methods work with parametric classes (e.g., `sort` for `List<T>`)
  3. Subtype: single interface to multiple representations of the same type (the most common type of polymorphism)

Polymorphism

- It comes in many flavours:
  1. Overloading: multiple implementations of the same method with different parameters
  2. Parametric – often used with *generics*: methods work with parametric classes (e.g., `sort` for `List<T>`)
  3. Subtype: single interface to multiple representations of the same type (the most common type of polymorphism)
- Subtype polymorphism: uses inheritance

Inheritance

- *Inheritance*

Inheritance

- *Inheritance*: reuse the code/features from a previously defined class

Inheritance

- *Inheritance*: reuse the code/features from a previously defined class
- Principle: DRY – “Do not repeat yourself”

Inheritance

- *Inheritance*: reuse the code/features from a previously defined class
- Principle: DRY – “Do not repeat yourself”
- If class `Child` inherits from class `Parent` then `Child` uses the same implementation as `Parent`...

Inheritance

- *Inheritance*: reuse the code/features from a previously defined class
- Principle: DRY – “Do not repeat yourself”
- If class `Child` inherits from class `Parent` then `Child` uses the same implementation as `Parent`...
- ... with some exceptions:

Inheritance

- *Inheritance*: reuse the code/features from a previously defined class
- Principle: DRY – “Do not repeat yourself”
- If class `Child` inherits from class `Parent` then `Child` uses the same implementation as `Parent`...
- ... with some exceptions:
  - *Shadowing*: `Child` can redefine data members (i.e, use the same name) from `Parent`

Inheritance

- *Inheritance*: reuse the code/features from a previously defined class
- Principle: DRY – “Do not repeat yourself”
- If class `Child` inherits from class `Parent` then `Child` uses the same implementation as `Parent`...
- ... with some exceptions:
  - *Shadowing*: `Child` can redefine data members (i.e, use the same name) from `Parent`
  - *Overriding*: `Child` can override methods defined in `Parent`

Inheritance

- *Inheritance*: reuse the code/features from a previously defined class
- Principle: DRY – “Do not repeat yourself”
- If class `Child` inherits from class `Parent` then `Child` uses the same implementation as `Parent`...
- ... with some exceptions:
  - *Shadowing*: `Child` can redefine data members (i.e, use the same name) from `Parent`
  - *Overriding*: `Child` can override methods defined in `Parent`
- Single vs. multiple inheritance

Subtype polymorphism

- Suppose that `Child` inherits `Parent`, and consider a function `SomeType foo(Parent p) {...}`

Subtype polymorphism

- Suppose that `Child` inherits `Parent`, and consider a function `SomeType foo(Parent p) {...}`
- In this case `foo` can be applied to an instance of *any* subclass of `Parent`:

```
Child child = new Child();
SomeType t = foo(child);
```

Subtype polymorphism

- Suppose that `Child` inherits `Parent`, and consider a function `SomeType foo(Parent p) {...}`
- In this case `foo` can be applied to an instance of *any* subclass of `Parent`:

```
Child child = new Child();
SomeType t = foo(child);
```

- Consider `Parent bar(Parent p) { return p; }`:

```
Child child = new Child();
Child p = bar(child);
```

Subtype polymorphism

- Suppose that `Child` inherits `Parent`, and consider a function `SomeType foo(Parent p) {...}`
- In this case `foo` can be applied to an instance of *any* subclass of `Parent`:

```
Child child = new Child();
SomeType t = foo(child);
```

- Consider `Parent bar(Parent p) { return p; }`:

```
Child child = new Child();
Child p = bar(child); // rejected by static type checker
```

Subtype polymorphism

- Suppose that `Child` inherits `Parent`, and consider a function `SomeType foo(Parent p) {...}`
- In this case `foo` can be applied to an instance of *any* subclass of `Parent`:

```
Child child = new Child();
SomeType t = foo(child);
```

- Consider `Parent bar(Parent p) { return p; }`:

```
Child child = new Child();
Child p = bar(child); // rejected by static type checker
Child p = (Child) bar(child);
```

Subtype polymorphism

- Suppose that `Child` inherits `Parent`, and consider a function `SomeType foo(Parent p) {...}`
- In this case `foo` can be applied to an instance of *any* subclass of `Parent`:

```
Child child = new Child();
SomeType t = foo(child);
```

- Consider `Parent bar(Parent p) { return p; }`:

```
Child child = new Child();
Child p = bar(child); // rejected by static type checker
Child p = (Child) bar(child); // dynamic check
```

Dynamic dispatch

- *Dynamic dispatch*

Dynamic dispatch

- *Dynamic dispatch*: select the correct implementation of a polymorphic method at runtime

Dynamic dispatch

- *Dynamic dispatch*: select the correct implementation of a polymorphic method at runtime

```
class Parent {... void f() {...} ...}
class Child : Parent {... void f() {...} ...}
...
Child child = new Child();
child.f()
```

Dynamic dispatch

- *Dynamic dispatch*: select the correct implementation of a polymorphic method at runtime

```
class Parent {... void f() {...} ...}
class Child : Parent {... void f() {...} ...}
...
Child child = new Child();
child.f() // f
```

Dynamic dispatch

- *Dynamic dispatch*: select the correct implementation of a polymorphic method at runtime

```
class Parent {... void f() {...} ...}
class Child : Parent {... void f() {...} ...}
...
Child child = new Child();
child.f() // f
((Parent) child).f();
```

Dynamic dispatch

- *Dynamic dispatch*: select the correct implementation of a polymorphic method at runtime

```
class Parent {... void f() {...} ...}
class Child : Parent {... void f() {...} ...}
...
Child child = new Child();
child.f() // f
((Parent) child).f(); // f
```

Late binding

- *Late binding*

Late binding

- *Late binding*:

```
class Parent {
   int a = 0;
   void f() { g(); }
   void g() { a = 1; }
}
class Child : Parent {
   int b = 0;
   void g() { b = 2; }
}
...
Child child = new Child();
child.f()
```

Late binding

- *Late binding*:

```
class Parent {
   int a = 0;
   void f() { g(); }
   void g() { a = 1; }
}
class Child : Parent {
   int b = 0;
   void g() { b = 2; }
}
...
Child child = new Child();
child.f() // b =
```

Late binding

- *Late binding*:

```
class Parent {
   int a = 0;
   void f() { g(); }
   void g() { a = 1; }
}
class Child : Parent {
   int b = 0;
   void g() { b = 2; }
}
...
Child child = new Child();
child.f() // b = 2;
```

Late binding

- *Late binding*:

```
class Parent {
   int a = 0;
   void f() { g(); }
   void g() { a = 1; }
}
class Child : Parent {
   int b = 0;
   void g() { b = 2; }
}
...
Child child = new Child();
child.f() // b = 2;a =
```

Late binding

- *Late binding*:

```
class Parent {
   int a = 0;
   void f() { g(); }
   void g() { a = 1; }
}
class Child : Parent {
   int b = 0;
   void g() { b = 2; }
}
...
Child child = new Child();
child.f() // b = 2;a = 0;
```

Interfaces vs. Abstract classes

- *Abstract class*

Interfaces vs. Abstract classes

- *Abstract class*: class without implementation for (at least one) methods

Interfaces vs. Abstract classes

- *Abstract class*: class without implementation for (at least one) methods
- *Interface*

Interfaces vs. Abstract classes

- *Abstract class*: class without implementation for (at least one) methods
- *Interface*: has methods without implementation
- Traits:
  1. Both can be inherited

Interfaces vs. Abstract classes

- *Abstract class*: class without implementation for (at least one) methods
- *Interface*: has methods without implementation
- Traits:
  1. Both can be inherited
  2. Abstract classes can have some implementations

Interfaces vs. Abstract classes

- *Abstract class*: class without implementation for (at least one) methods
- *Interface*: has methods without implementation
- Traits:
  1. Both can be inherited
  2. Abstract classes can have some implementations
  3. Methods in interfaces must be implemented by the implementing class

Related concepts

- Generics
- Packages
- Modules
- Namespaces

Facts

- OOP scales; it is widely used in the industry
- Not all languages that are advertised as OOP languages support all the features above (e.g., C++ does not have GC)
- Sometimes, misunderstanding of the OOP concepts may complicate your code

Resources

- Chapter 10: `http://websrv.dthu.edu.vn/attachments/newsevents/content2415/Programming_Languages_-_Principles_and_Paradigms_thereds1106.pdf`
- KOOL: `https://github.com/kframework/k/tree/master/k-distribution/tutorial/2_languages/2_kool`